The Reagent Language

DURWASA CHAKRABORTY, TODO: institution, India

TODO

Additional Key Words and Phrases: concurrency, transactional memory, operational semantics, internal consistency

1 Abstract syntax

n	$\in$	$\mathbb{N}$	natural-number literals
l	$\in$	Loc	shared mutable locations
x	$\in$	Var	immutable, let-bound variables
v	::=	$n \mid () \mid \mathsf{true} \mid \mathsf{false}$	values (naturals, unit, booleans)
c	::=	$\mathsf{get}\ (l)$	read a shared location
	$\mid$	$\mathsf{set}\ (l, e)$	write a shared location
	$\mid$	begin	open a transaction
	$\mid$	end	commit the current transaction
	$\mid$	retry	abort and restart the transaction
e	::=	$v \mid x$	values, variables
	$\mid$	$e_1 + e_2$	addition (the whiteboard's $\oplus$)
	$\mid$	$\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2$	binding
	$\mid$	c	a command (and not a control-flow)
	$\mid$	$\mathsf{if}\ e\ \mathsf{then}\ e\ \mathsf{else}\ e$	conditional

1.1 Semantics :: of the Reagent Language

Each configuration C in Φ represents the global state of the program at a particular point during execution. Thread identifiers form a fixed finite set $\mathit{ThId} \subseteq \mathbb{N}$, and transaction identifiers a set $\mathit{TxId} = \{t_1, t_2, \dots\}$. Formally, a configuration is a six-tuple $C = \langle \sigma, \lambda, \theta, E, \hat{\sigma}, \hat{E} \rangle$, where:

- $\sigma : \mathit{Loc} \to \mathbb{N}$ maps each shared memory location to its current value.
- $\lambda \in \mathit{ThId} \cup \{\bot\}$ records the *thread* that currently holds the lock. $\lambda = i$ means thread i owns the live transaction and is the only thread permitted to take a transactional step; $\lambda = \bot$ means no thread is active.
- $\theta \in \mathit{TxId} \cup \{\bot\}$ is the identifier of the live transaction, minted *fresh* by each begin; $\theta = \bot$ means no transaction is active. It supplies the tag on the emitted actions — when $\theta = t$ the labels read $\mathsf{get}(l, v)\ @\ t$, $\mathsf{end}\ @\ t$, and so on.
- $E : \mathit{ThId} \to \mathit{Expr}$ maps each thread to the expression it is currently evaluating. Since $\mathit{dom}(E)$ is fixed (Section 2), E is a total map.
- $\hat{\sigma} : \mathit{Loc} \to \mathbb{N}$ is the snapshot of the store taken when the live transaction began, used to roll the store back on restart.
- $\hat{E} \in \mathit{Expr}$ is the expression from which execution resumes when a retry is encountered.

Author's Contact Information: Durwasa Chakraborty, durwasa.chakraborty@gmail.com, TODO: institution, India.

The snapshot $\hat{\sigma}$ and saved body $\hat{E}$ matter only inside a transaction, but we do not tag them with "$\lambda \neq \bot$" each time: an initial configuration already gives them values (Section 2) — a run starts with $\lambda = \theta = \bot$ (no thread, no transaction) and $\hat{\sigma} = \sigma_0$, $\hat{E} = ()$ — and every rule keeps them defined thereafter.

In every rule the *active* thread is i (the one taking the step) with $E(i) = \mathsf{ECtx}[\![\, r \,]\!]$, where r is the unique redex its evaluation context ECtx locates (the subterm currently able to step). There is no scheduler: the nondeterminism is inherent in $\rightarrow$, and any thread with an enabled redex may be the one that steps. Every transactional rule (Get, Set, End, Retry) carries the premise $\lambda = i$: *only the thread that owns the live transaction may step inside it.* The conclusion rewrites that redex in place and *frames* the rest of the table, $E' = E[i \mapsto \mathsf{ECtx}[\![\, \cdot \,]\!]]$ with $E'(j) = E(j)$ for every $j \neq i$; the label above the arrow is tagged with the live transaction id θ (the third component of the configuration). Figure 1 gives a

$$\textsc{Get}\ \frac{E(i) = \mathsf{ECtx}[\![\, \mathsf{get}\ (l) \,]\!] \qquad \lambda = i \qquad \sigma(l) = v}{\langle \sigma, \lambda, \theta, E, \hat{\sigma}, \hat{E} \rangle \xrightarrow{\mathsf{get}(l,v)\ @\ \theta} \langle \sigma, \lambda, \theta, E[i \mapsto \mathsf{ECtx}[\![\, v \,]\!]], \hat{\sigma}, \hat{E} \rangle}$$

$$\textsc{Set}\ \frac{E(i) = \mathsf{ECtx}[\![\, \mathsf{set}\ (l, v) \,]\!] \qquad \lambda = i}{\langle \sigma, \lambda, \theta, E, \hat{\sigma}, \hat{E} \rangle \xrightarrow{\mathsf{set}(l,v)\ @\ \theta} \langle \sigma[l \mapsto v], \lambda, \theta, E[i \mapsto \mathsf{ECtx}[\![\, () \,]\!]], \hat{\sigma}, \hat{E} \rangle}$$

$$\textsc{Begin}\ \frac{E(i) = \mathsf{ECtx}[\![\, \mathsf{begin}\,;\, e \,]\!] \qquad \lambda = \bot \qquad \theta\ \mathrm{fresh}}{\langle \sigma, \bot, \bot, E, \hat{\sigma}, \hat{E} \rangle \xrightarrow{\mathsf{begin}\ @\ \theta} \langle \sigma, i, \theta, E[i \mapsto \mathsf{ECtx}[\![\, () \,]\!]], \sigma, e \rangle}$$

$$\textsc{End}\ \frac{E(i) = \mathsf{ECtx}[\![\, \mathsf{end} \,]\!] \qquad \lambda = i}{\langle \sigma, \lambda, \theta, E, \hat{\sigma}, \hat{E} \rangle \xrightarrow{\mathsf{end}\ @\ \theta} \langle \sigma, \bot, \bot, E[i \mapsto \mathsf{ECtx}[\![\, () \,]\!]], \hat{\sigma}, \hat{E} \rangle}$$

$$\textsc{Retry}\ \frac{E(i) = \mathsf{ECtx}[\![\, \mathsf{retry} \,]\!] \qquad \lambda = i}{\langle \sigma, \lambda, \theta, E, \hat{\sigma}, \hat{E} \rangle \xrightarrow{\mathsf{retry}\ @\ \theta} \langle \hat{\sigma}, \bot, \bot, E[i \mapsto \mathsf{ECtx}[\![\, \mathsf{begin}\,;\, \hat{E} \,]\!]], \hat{\sigma}, \hat{E} \rangle}$$

Fig. 1. Semantics of Reagent commands. Each rule fires on the redex of the active thread i; the transactional rules require $\lambda = i$ (only the owner thread may step), and the emitted label is tagged with the live transaction id θ. Unlisted six-tuple fields are unchanged.

formal description of the transition rules. Get reads the value stored at a shared location l. If the active thread i owns the live transaction ($\lambda = i$) and $\sigma(l) = v$, then get (l) returns the value v; the emitted action $\mathsf{get}(l, v)\ @\ \theta$ records both the value v read and the id θ of the live transaction. The store and transaction state remain unchanged.

Set updates a shared location. When the owner thread i (with $\lambda = i$) executes set (l, v), the value stored at location l is updated to v and the command returns the unit value $()$; the emitted action $\mathsf{set}(l, v)\ @\ \theta$ records the written value v and the transaction id θ.

Begin starts a transaction. No transaction may currently be alive ($\lambda = \bot$); transactions are therefore mutually exclusive, with at most one live transaction across all threads at any time. Begin mints a *fresh* transaction id θ (the new live id), records the executing thread as the owner by setting $\lambda := i$, and saves the current store and continuation in $\hat{\sigma}$ and $\hat{E}$ respectively.

End commits the current transaction, resetting both the owner λ and the live transaction id θ to $\bot$. The store is unchanged.

Retry is retrying with *lock-releasing*. Instead of keeping the thread inside the transaction and spinning, it (i) rolls the store back to the snapshot $\hat{\sigma}$, so the abandoned attempt has no effect; (ii) sets $\lambda = \bot$ and clears the live transaction

id, *releasing the single critical section* so that another thread may now begin and run; and (iii) re-arms $E(i)$'s expression as begin ; $\hat{E}$, so that when i next steps it must re-acquire the lock through Begin before re-running the saved body.

Because every transactional rule requires $\lambda = i$, a thread can only read, write, commit, or retry inside a transaction it opened itself. A get (l), set (l, e), end, or retry reached with $\lambda \neq i$ — in particular one with no matching enclosing begin — has no rule to fire, so the run gets stuck rather than taking an ill-formed step. Well-formedness is thus enforced by the operational semantics itself, not assumed (Section 3).

1.2 Evaluation contexts

An evaluation context ECtx is an expression containing a single hole $[\cdot]$ that marks exactly where the next reduction step occurs. We give the evaluation context its own metavariable ECtx precisely to keep it separate from E, which represents a mapping from thread identifiers (*Tid*) to expressions (*Expr*); the two are distinct objects and should not be conflated.

$$\mathsf{ECtx} \;::=\; [\cdot] \mid \mathsf{ECtx}\,;\, e \mid \mathsf{let}\ x = \mathsf{ECtx}\ \mathsf{in}\ e \mid \mathsf{ECtx}{+}e \mid v{+}\mathsf{ECtx} \mid \mathsf{ECtx} == e \mid v == \mathsf{ECtx} \mid \mathsf{if}\ \mathsf{ECtx}\ \mathsf{then}\ e\ \mathsf{else}\ e \mid \mathsf{set}\,(l, \mathsf{ECtx}).$$

We write $\mathsf{ECtx}[\![\, e \,]\!]$ for the expression obtained by plugging e into the hole. Thus $E(t) = \mathsf{ECtx}[\![\, \mathsf{begin} \,]\!]$ asserts that the redex thread t is about to execute is begin, sitting inside the surrounding continuation ECtx.

1.3 The command rules in context form

Every command rule now fires on its redex inside an arbitrary context ECtx, preserving the continuation. The configuration is the six-tuple $\langle \sigma, \lambda, \theta, E, \hat{\sigma}, \hat{E} \rangle$ of Section 1.1.

- $E(i) = \mathsf{ECtx}[\![\, \mathsf{begin} \,]\!]$ and $E' = E[i \mapsto \mathsf{ECtx}[\![\, () \,]\!]]$: the begin reduces to $()$ *in place*, leaving the continuation ECtx.
- $\hat{E}' = \mathsf{ECtx}[\![\, () \,]\!]$: the saved resume point is $\mathsf{ECtx}[\![\, () \,]\!]$, the body *after* begin, which is precisely what Retry reinstates.

Retry rolls the store back to the snapshot $\hat{\sigma}$, releases the lock ($\lambda = \bot$), and re-arms $E(i)$ as begin ; $\hat{E}$ so that the thread must re-acquire the lock through Begin before re-running.

1.4 Administrative reductions

$$\mathsf{ECtx}[\![\, \mathsf{let}\ x = v\ \mathsf{in}\ e \,]\!] \longrightarrow \mathsf{ECtx}[\![\, e[x \mapsto v] \,]\!] \qquad \mathsf{ECtx}[\![\, () \,;\, e \,]\!] \longrightarrow \mathsf{ECtx}[\![\, e \,]\!]$$

$$\mathsf{ECtx}[\![\, n_1 + n_2 \,]\!] \longrightarrow \mathsf{ECtx}[\![\, n \,]\!] \quad (n = n_1 + n_2) \qquad \mathsf{ECtx}[\![\, \mathsf{if\ true\ then}\ e_1\ \mathsf{else}\ e_2 \,]\!] \longrightarrow \mathsf{ECtx}[\![\, e_1 \,]\!]$$

Here the notation is substitution $e[x \mapsto v]$ in the let rather than map update.

2 Initial configuration and termination

The transition system $\mathcal{S} = (\Phi, \longrightarrow)$ has an *infinite* state space: a configuration $\langle \sigma, \lambda, \theta, E, \hat{\sigma}, \hat{E} \rangle$ ranges over all stores, all thread maps, all snapshots, and all saved bodies, and there are infinitely many of each. A program does not fix a single starting point; it determines a *set* of initial configurations, and a run may begin from any one of them.

2.1 Initial configurations

The set of *initial configurations* is

$$\mathcal{I} \;=\; \big\{\, \langle\, \sigma_0,\ \bot,\ \bot,\ E_0,\ \sigma_0,\ ()\,\rangle \;:\; \sigma_0, E_0 \text{ meet the conditions below} \,\big\} \;\subseteq\; \Phi,$$

and a run may start from any $C_0 \in \mathcal{I}$.

- *Locations are fixed and finite.* Loc is a fixed finite set and $\sigma_0 : Loc \to \mathbb{N}$ is any initial store: $\forall l \in Loc.\ \sigma_0(l) = n$ for some $n \in \mathbb{N}$. So $\mathcal{I}$ varies with σ_0.
- *Threads are fixed and finite.* $dom(E_0) = ThId$ is a fixed finite set of thread identifiers, and $E_0 : ThId \to Expr$ is the initial program, mapping each thread to its starting expression. No thread is ever created or destroyed, so $dom(E) = ThId$ at every configuration.
- *Every thread starts with a real program.* $E_0(i) \neq ()$ for each $i \in ThId$: a thread already at the unit value is finished and cannot step, so an initial configuration must give every thread an unreduced expression for the system to make progress.
- *Nothing is live yet.* $\lambda = \bot$ (no thread owns the lock) and $\theta = \bot$ (no transaction is live), since before any begin no thread is in a transaction. The snapshot and saved body are given values here, $\hat{\sigma}_0 = \sigma_0$ and $\hat{E}_0 = ()$, so the configuration is fully defined from the outset.

Fixing any $C_0 \in \mathcal{I}$, the configurations that matter are those reachable from it,

$$\mathrm{Reach}(C_0) \;=\; \{\, C \;:\; C_0 \to^* C \,\},$$

where $\to^*$ is the reflexive–transitive closure of $\to$. There is no scheduler: the nondeterminism of $\to$ is inherent, and at each configuration *any* thread whose redex is enabled may take the next step. These reachable step sequences are the program's *executions*.

2.2 Termination

We do not define a "final" or "terminal" configuration; the transition relation has no distinguished stopping states. We instead *assume* that a run halts after finitely many steps, and record the condition under which it has run to completion: every thread has reduced to the unit value,

$$E(i) = () \qquad \forall i \in dom(E).$$

Here $()$ is the unit *value* of the abstract syntax, so this is a pointwise statement $E(i) = ()$; writing $E : ThId \rightharpoonup \{()\}$ would be a category error, since the grammar produces values, not sets.

3 Worked example: well-formedness is structural

Because λ now names the *owner thread* and every transactional rule carries the premise $\lambda = i$, a thread can take a transactional step only inside a transaction it has itself opened. Call a program *well-formed* when every get (l), set (l, e), end, or retry it reaches sits inside a matching enclosing begin . . . end. The point of this section is that well-formedness is *structural*: an ill-formed redex simply has no rule to fire, so the run gets stuck rather than performing the action. We never need well-formedness as a separate side condition.

Conventions. There is a single shared location l; we write $[\, l \mapsto n \,]$ for the store with $\sigma(l) = n$. The configuration is the six-tuple $\langle \sigma, \lambda, \theta, E, \hat{\sigma}, \hat{E} \rangle$ of Section 1.1, with λ the owner thread and θ the live transaction id. Thread identifiers are $1, 2 \in ThId$; E is shown as a map $\{1{:}e_1,\ 2{:}e_2\}$. Every run starts from $C_0 = \langle [\, l \mapsto 0 \,],\ \bot,\ \bot,\ E_0,\ [\, l \mapsto 0 \,],\ () \rangle$.

3.1 An ill-formed get cannot fire

Let thread 1 run the ill-formed program $P \equiv \mathsf{get}\ (l)$ — a read with no enclosing begin. The initial configuration is

$$C_0 \;=\; \langle [\, l \mapsto 0 \,],\ \bot,\ \bot,\ \{1{:}\mathsf{get}\ (l)\},\ [\, l \mapsto 0 \,],\ () \rangle.$$

The active thread is 1 and its redex is get (l) (context $\mathsf{ECtx} = [\cdot]$). We check every rule of Figure 1:

- Get requires $\lambda = 1$, but $\lambda = \bot$. *Fails.*
- Begin requires the redex to be $\mathsf{begin}\,;\, e$, but it is get (l). *Fails.*
- Set, End, Retry require their own redexes (set (l, v), end, retry) and $\lambda = 1$; both premises fail.

No rule matches, so C_0 takes *no* step:

$$\langle [\, l \mapsto 0 \,],\ \bot,\ \bot,\ \{1{:}\mathsf{get}\ (l)\},\ [\, l \mapsto 0 \,],\ () \rangle \;\not\twoheadrightarrow .$$

The ill-formed get (l) never executes; the thread is stuck. Contrast the well-formed version, $\mathsf{begin}\,;\, (\mathsf{get}\ (l)\,;\, \mathsf{end})$. There the leading begin fires first (setting $\lambda := 1$), and only *then*, with $\lambda = 1$, can the get (l) fire: it is exactly the begin that grants the ownership get (l) demands.

3.2 A thread cannot act inside another thread's transaction

This is the case the premise $\lambda = i$ is really designed to rule out. Take two threads,

$$E_0 \;=\; \{1{:}\mathsf{begin}\,;\, (\mathsf{get}\ (l)\,;\, \mathsf{end}),\ 2{:}\mathsf{get}\ (l)\},$$

so thread 1 is well-formed and thread 2 tries a bare get (l). Let thread 1 open its transaction:

$$\begin{aligned} &\langle [\, l \mapsto 0 \,],\ \bot,\ \bot,\ E_0,\ [\, l \mapsto 0 \,],\ () \rangle \\ &\quad \xrightarrow{\mathsf{begin}\,@\,t_1} \langle [\, l \mapsto 0 \,],\ 1,\ t_1,\ \{1{:}() \,;\, (\mathsf{get}\ (l)\,;\, \mathsf{end}),\ 2{:}\mathsf{get}\ (l)\},\ [\, l \mapsto 0 \,],\ \mathsf{get}\ (l)\,;\, \mathsf{end} \rangle. \end{aligned}$$

Now $\lambda = 1$: thread 1 owns the live transaction t_1. Suppose thread 2 is the next to step; its redex is get (l). The only rule that could fire is Get, and it requires $\lambda = 2$ — but $\lambda = 1$. The premise fails, so thread 2's read is *not enabled*:

$$\text{there is no } C' \text{ with} \quad \langle [\, l \mapsto 0 \,],\ 1,\ t_1,\ \{1{:}\ldots,\ 2{:}\mathsf{get}\ (l)\},\ [\, l \mapsto 0 \,],\ \mathsf{get}\ (l)\,;\, \mathsf{end} \rangle \xrightarrow{\mathsf{get}(l,0)\,@\,\cdot} C'.$$

Under the old rule (premise $\lambda = t$, with t unconstrained by the acting thread) this step *would* have fired, letting thread 2 read inside thread 1's transaction and mislabel it $@t_1$. The premise $\lambda = i$ removes that: only the owner may step, so thread 2 must wait until thread 1 commits (λ returns to $\bot$) and then open its own transaction. Well-formedness holds by construction.

4 Histories and transactions

A run emits a sequence of runtime actions

$$a \;::=\; \mathsf{begin}\,@\,t \;\mid\; \mathsf{end}\,@\,t \;\mid\; \mathsf{retry}\,@\,t \;\mid\; \mathsf{get}(l, v)\,@\,t \;\mid\; \mathsf{set}(l, v)\,@\,t.$$

An *event* is a pair $e = (\iota, a)$ pairing a unique identifier $\iota \in \mathit{EventId}$ with an action a; we write $\mathsf{act}(e) = a$ and $\mathsf{tid}(e) = t$ for the transaction identifier carried by a.

Definition 4.1 (History). A *history* is a pair $H = (E, po)$, where E is a finite set of events with distinct identifiers and po (the *program order*) is a total order on E. The history of a run collects the events it emits, ordered by po as they occur.

Definition 4.2 (Transaction). Let $H = (E, po)$ and let t be a transaction identifier occurring in E. The *transaction* t is the pair

$$T_t \;=\; (E_t, po_t), \qquad E_t = \{\, e \in E : \mathrm{tid}(e) = t \,\}, \qquad po_t = po \upharpoonright E_t = po \cap (E_t \times E_t),$$

i.e. E_t is the events of H tagged t, and po_t is *po restricted* to them: viewing the order po as a set of pairs, $po \upharpoonright E_t$ keeps only the pairs whose *both* events lie in E_t. So po_t is the same ordering filtered to t's own events; being the restriction of a total order, it is itself total.

Definition 4.3 (Committed transaction). A transaction $T_t = (E_t, po_t)$ is *committed* if its po_t-greatest event is a commit, $\mathrm{act}\big(\max_{po_t} E_t\big) = \mathsf{end} \mathbin{@} t$ — it is opened by $\mathsf{begin} \mathbin{@} t$ and closed by $\mathsf{end} \mathbin{@} t$. It is *aborted* if instead $\mathrm{act}\big(\max_{po_t} E_t\big) = \mathsf{retry} \mathbin{@} t$, and *pending* if E_t contains neither $\mathsf{end} \mathbin{@} t$ nor $\mathsf{retry} \mathbin{@} t$ (the run stopped inside it).

Example 4.4. Take a run on a single location l (initially 0) in which the lock serialises three transactions; number the events by position, $e_k = (k, a_k)$. The history is $H = (E, po)$ with $E = \{e_1, \ldots, e_{11}\}$ and $po : e_1 < e_2 < \cdots < e_{11}$, where

$$\begin{aligned} e_1 \ldots e_4 &: \ \mathsf{begin} \mathbin{@} t_1 \cdot \mathsf{get}(l, 0) \mathbin{@} t_1 \cdot \mathsf{set}(l, 1) \mathbin{@} t_1 \cdot \mathsf{end} \mathbin{@} t_1, \\ e_5 \ldots e_7 &: \ \mathsf{begin} \mathbin{@} t_2 \cdot \mathsf{get}(l, 1) \mathbin{@} t_2 \cdot \mathsf{retry} \mathbin{@} t_2, \\ e_8 \ldots e_{11} &: \ \mathsf{begin} \mathbin{@} t_3 \cdot \mathsf{get}(l, 1) \mathbin{@} t_3 \cdot \mathsf{set}(l, 2) \mathbin{@} t_3 \cdot \mathsf{end} \mathbin{@} t_3. \end{aligned}$$

(t_1 commits $l = 1$; t_2 reads it and retries, so its thread re-runs as the fresh identifier t_3, which commits $l = 2$.)

A single *event* is $e_2 = (2,\ \mathsf{get}(l, 0) \mathbin{@} t_1)$, with $\mathrm{tid}(e_2) = t_1$ and $\mathrm{act}(e_2) = \mathsf{get}(l, 0) \mathbin{@} t_1$ — a read of 0 from l in t_1.

Projecting H by transaction identifier gives three *transactions*:

$$T_{t_1} = (\{e_1, e_2, e_3, e_4\},\ po_{t_1}), \quad T_{t_2} = (\{e_5, e_6, e_7\},\ po_{t_2}), \quad T_{t_3} = (\{e_8, e_9, e_{10}, e_{11}\},\ po_{t_3}),$$

each with $po_t = po \upharpoonright E_t$. Checking the po_t-greatest event: $\mathrm{act}(\max_{po_{t_1}} E_{t_1}) = \mathsf{end} \mathbin{@} t_1$ and $\mathrm{act}(\max_{po_{t_3}} E_{t_3}) = \mathsf{end} \mathbin{@} t_3$, so T_{t_1} and T_{t_3} are *committed*; whereas $\mathrm{act}(\max_{po_{t_2}} E_{t_2}) = \mathsf{retry} \mathbin{@} t_2$, so T_{t_2} is aborted, not a committed transaction.

5 Internal consistency

Cerone et al.'s internal-consistency axiom is

$$\forall (E, po) \in H.\, \forall e \in E.\, \forall x, n. \quad \big(e = (_,\ \mathsf{read}(x, n)) \wedge po^{-1}(e) \cap HEvent_x \neq \emptyset\big) \implies \max_{po}\big(po^{-1}(e) \cap HEvent_x\big) = (_,\ _(x, n)), \tag{Int}$$

where $HEvent_x$ is the set of read and write events on x, and $(_, _(x, n))$ is a read or a write of n on x.

Example 5.1 (Int needs no commit). Neither transaction below commits, yet Int still pins both reads:

$$H : \quad \mathsf{begin} \mathbin{@} t_1 \cdot \mathsf{set}(x, 2) \mathbin{@} t_1 \cdot \mathsf{get}(x, 2) \mathbin{@} t_1 \cdot \mathsf{retry} \mathbin{@} t_1 \cdot \mathsf{begin} \mathbin{@} t_2 \cdot \mathsf{set}(x, 5) \mathbin{@} t_2 \cdot \mathsf{get}(x, 5) \mathbin{@} t_2$$

(t_1 aborts, t_2 is left pending). For $\mathsf{get}(x, 2) \mathbin{@} t_1$, $po^{-1}(e) \cap HEvent_x = \{\mathsf{set}(x, 2) \mathbin{@} t_1\}$, whose $\max_{po}$ is a write of 2 on x, so the read is forced to 2; likewise $\mathsf{get}(x, 5) \mathbin{@} t_2$ is forced to 5. Committedness (if that is even a word) never entered.

Example 5.2 (external read: Int vacuous). When a read is a transaction's first access to x, Int is silent:

$$H : \quad \mathsf{begin} \mathbin{@} t_1 \cdot \mathsf{set}(x, 7) \mathbin{@} t_1 \cdot \mathsf{end} \mathbin{@} t_1 \cdot \mathsf{begin} \mathbin{@} t_2 \cdot \mathsf{get}(x, 7) \mathbin{@} t_2$$

(t_1 committed). Here $\mathsf{get}(x, 7) \mathbin{@} t_2$ has $po^{-1}(e) \cap HEvent_x = \emptyset$, so the premise fails and Int holds *vacuously*.

6 External consistency

Internal consistency (Section 5) constrains a read only when the same transaction has already touched that location — when $po^{-1}(e) \cap HEvent_x \neq \emptyset$. Every other read is one whose value *cannot* come from within its own transaction: a read of x not preceded, in its transaction, by any access to x. We call such a read *external*, because the value it returns must come from *another* transaction. External reads are exactly the ones Int leaves free — its premise is vacuous, as in the last example of Section 5 — and external consistency is what fixes them.

Following Cerone, we summarise a transaction's effect on a location x with two judgements. $T \vdash \mathsf{Write}\ x : n$ holds when T writes x and its po-last write to x has value n — the value T leaves in x. And $T \vdash \mathsf{Read}\ x : n$ holds when T's po-first access to x is a read returning n — precisely an *external* read of x by T.

The value such a read may return comes from the committed transactions that took effect before it. That order is *arbitration*.

Definition 6.1 (Arbitration). For committed transactions with identifiers $s \neq t$ in a history $H = (E, po)$,

$$T_s <_{\mathrm{AR}} T_t \quad \text{iff} \quad \mathsf{end}\ @\ s \text{ precedes } \mathsf{begin}\ @\ t \text{ in } po,$$

i.e. s commits before t opens. Since the lock keeps at most one transaction live, committed transactions never overlap, so $<_{\mathrm{AR}}$ totally orders the committed transactions of H — it is their *commit order*. Write $\mathrm{AR}^{-1}(T) = \{\, S \mid S <_{\mathrm{AR}} T \,\}$ for the committed transactions preceding T.

We do *not* model a separate visibility relation. VIS would record that one transaction can *see* another's effects — a distinct notion only under weaker isolation, where serializability fails, visibility is strictly weaker than arbitration (VIS $\subsetneq$ AR), and VIS must be given on its own. Here there is no such gap: every committed transaction's writes are available to every AR-later transaction, so arbitration alone suffices.

External consistency then says: an external read returns the value written by the AR-latest committed writer of that location, or the initial value 0 if there is none. Writing the writers S over *committed* transactions only,

$$\begin{aligned} &\forall T \in H.\ \forall x, n.\quad T \vdash \mathsf{Read}\ x : n \implies \\ &\quad \big(\mathrm{AR}^{-1}(T) \cap \{\, S \mid S \vdash \mathsf{Write}\ x : _ \,\} = \emptyset \ \wedge\ n = 0\big) \\ &\quad \vee\ \max_{\mathrm{AR}}\big(\mathrm{AR}^{-1}(T) \cap \{\, S \mid S \vdash \mathsf{Write}\ x : _ \,\}\big) \vdash \mathsf{Write}\ x : n. \end{aligned} \tag{Ext}$$

Example 6.2 (committed writer is visible). t_1 commits its write, so Ext pins t_2's read to it:

$$H: \quad \mathsf{begin}\ @\ t_1 \cdot \mathsf{set}(x, 7)\ @\ t_1 \cdot \mathsf{end}\ @\ t_1 \cdot \mathsf{begin}\ @\ t_2 \cdot \mathsf{get}(x, 7)\ @\ t_2$$

$\mathsf{end}\ @\ t_1$ precedes $\mathsf{begin}\ @\ t_2$, so $T_{t_1} <_{\mathrm{AR}} T_{t_2}$, and T_{t_1} is committed with $T_{t_1} \vdash \mathsf{Write}\ x : 7$. Hence $\max_{\mathrm{AR}}\big(\mathrm{AR}^{-1}(T_{t_2}) \cap \{S \mid S \vdash \mathsf{Write}\ x : _\}\big) = T_{t_1}$, and Ext forces the read to 7.

Example 6.3 (no committed writer: read is 0). t_1 aborts, so its write is uncommitted; Ext still holds — with the initial value:

$$H: \quad \mathsf{begin}\ @\ t_1 \cdot \mathsf{set}(x, 2)\ @\ t_1 \cdot \mathsf{retry}\ @\ t_1 \cdot \mathsf{begin}\ @\ t_2 \cdot \mathsf{get}(x, 0)\ @\ t_2$$

T_{t_1} is not committed, so it is not in $\mathrm{AR}^{-1}(T_{t_2})$; hence $\mathrm{AR}^{-1}(T_{t_2}) \cap \{S \mid S \vdash \mathsf{Write}\ x : _\} = \emptyset$, and Ext forces the read to 0 — matching the rolled-back store.